CSA0612-DESIGN AND ANALYSIS OF ALGORITHMS.

CO1-ASSESSMENT TOOL 2-CONCEPT MAPPING.

Question 75 Concept Map: Algorithm → Recursion → Recursion Tree → Level Cost → Total Cost → Time Complexity.

1. Visualizing via Recursion Tree & Cost Distribution:

When a recursive algorithm executes (such as a Divide and Conquer algorithm like Merge Sort), it breaks a problem of size n into smaller subproblems. A recursion tree is a graphical representation of this process where:

- Each node represents the cost of a subproblem processed at a given recursive stage.
- The execution is broken down into tiers or levels, where the cost at any single level is the sum of all subproblem costs active at that depth.

2. Extended Map: Branching Factor (b) and Tree Height (h):

To accurately analyze the tree, we incorporate two core structural parameters:

- **Branching Factor (b):** The number of children each parent node produces (i.e., how many independent subproblems a single problem splits into, such as $b = 2$ in standard Merge Sort).
- **Tree Height (h):** The maximum number of recursive steps required to reach the base case. The height is fundamentally constrained by the input size n and branching factor, typically scaling as $h = \log_b n$.

3. Summation of Level Costs to Determine Total Cost:

The final execution time (Total Cost) of the recursive algorithm is derived by calculating the cumulative sum of all level costs from the root node ($l = 0$) down to the leaf nodes ($l = h$):

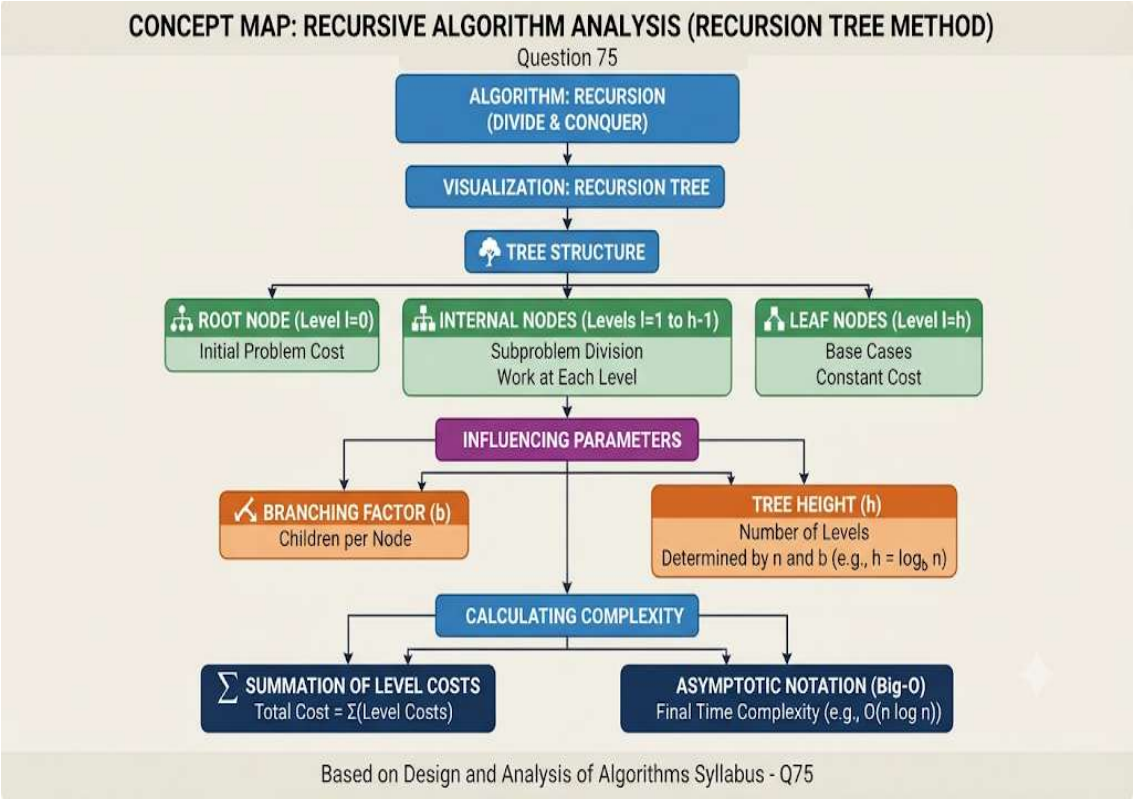
$$\text{Total Cost} = \sum_{l=0}^{h} (\text{Cost at level } l)$$

- If work is distributed evenly across all levels (e.g., $O(n)$ work per level across $\log n$ levels), the total complexity evaluates to $O(n \log n)$.

4. Interpretive Significance:

This mapping framework simplifies algorithm analysis by translating complex, non-linear recurrence relations into standard geometric or arithmetic series. It allows developers and analysts to isolate where computational bottlenecks occur—whether at the root, uniformly across levels, or concentrated at the leaf base cases.

Diagram:



Question 76 Concept Map: Algorithm → Loop Structure → Iterations → Operation Count → Time Complexity.

1. Iterative Constructs & Operation Accumulation:

Iterative algorithms rely on explicit looping constructs (such as for and while) to execute code sequences repeatedly. Every execution of an inner statement contributes to the cumulative operation count. As input data scales, the total frequency of these operations dictates the algorithm's running time efficiency.

2. Extended Branches: Single Loop vs. Nested Loops:

The structural complexity of iterative code branches into distinct patterns:

- **Single Loop (Linear Scaling):** A single loop iterating from 1 to n executes its inner statements exactly n times. This direct linear mapping yields an operation growth rate of $O(n)$.
- **Nested Loops (Polynomial Scaling):** A loop placed inside another loop (e.g., an outer loop running n times, and for every iteration, an inner loop running n times) multiplies the iteration space. The total

operations scale to $n \times n = n^2$, resulting in a polynomial time complexity of $O(n^2)$.

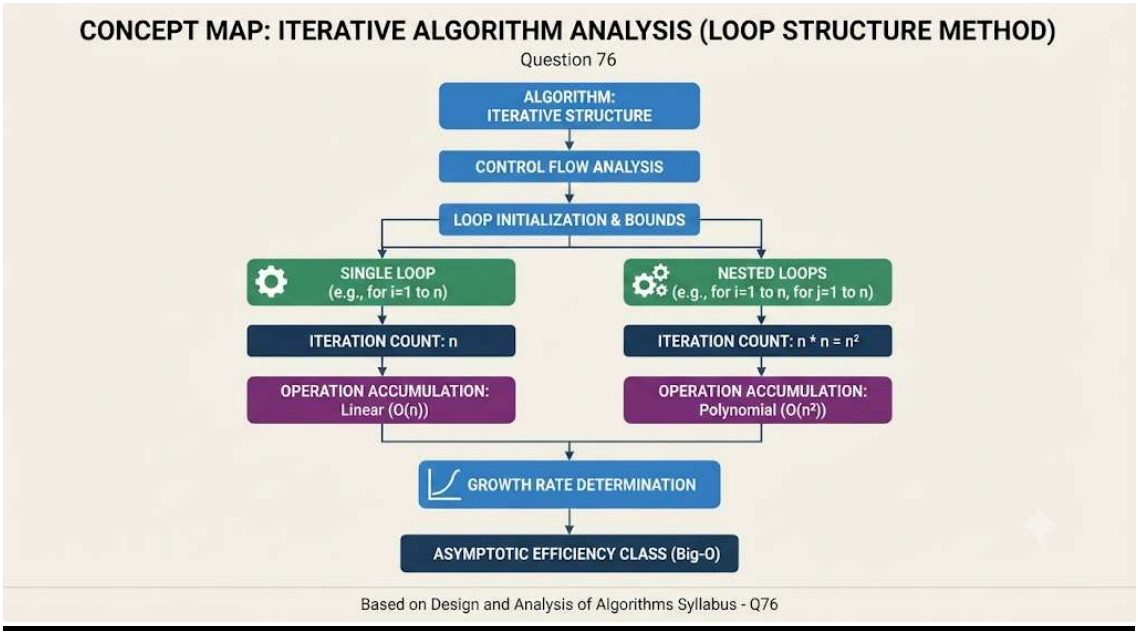
3. Iterations Determining Growth Rate:

The mathematical relationship between input size n and the loop control bounds directly determines the asymptotic growth rate. If loop increments scale additively, growth is linear; if loop iterations multiply across dimensions (such as triangular iterations or multi-dimensional matrices), growth scales polynomially or exponentially.

4. Interpretive Significance:

This mapping provides a direct, predictable pathway for evaluating iterative code. By isolating loop structures, tracking control variable bounds, and converting them into mathematical summations, analysts can accurately forecast performance bottlenecks and check code scalability prior to deployment on large-scale datasets.

Diagram:


CONCEPT MAP: ITERATIVE ALGORITHM ANALYSIS (LOOP STRUCTURE METHOD)
Question 76
ALGORITHM: ITERATIVE STRUCTURE
CONTROL FLOW ANALYSIS
LOOP INITIALIZATION & BOUNDS
SINGLE LOOP (e.g., for i=1 to n)
NESTED LOOPS (e.g., for i=1 to n, for j=1 to n)
ITERATION COUNT: n
ITERATION COUNT: n * n = n²
OPERATION ACCUMULATION: Linear (O(n))
OPERATION ACCUMULATION: Polynomial (O(n²))
GROWTH RATE DETERMINATION
ASYMPTOTIC EFFICIENCY CLASS (Big-O)
Based on Design and Analysis of Algorithms Syllabus - Q76
